

Introdução

Notas organizadas a partir dos slides e das anotações manuscritas de aula, Setembro/2026.

Disciplina: Métodos Avançados de Programação (MAP) — UEPB, 26.2

Professora: Luciana de Queiroz Leal Gomes

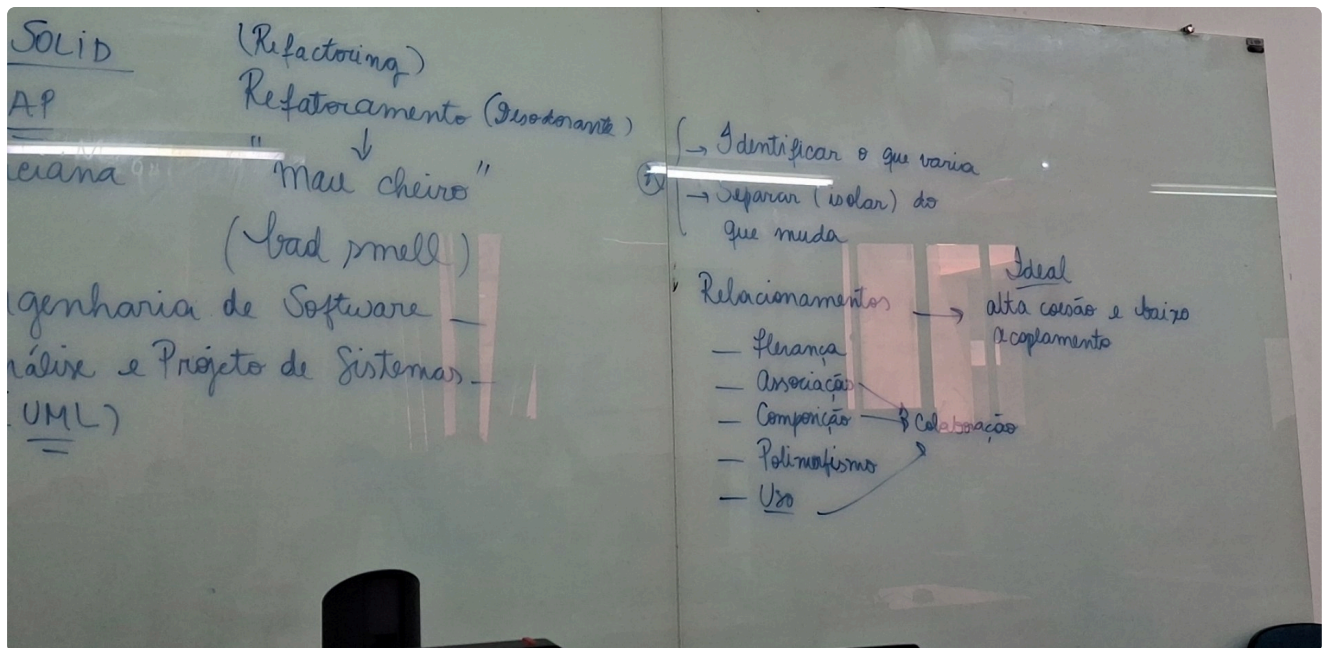
Referências: Larman (Utilizando UML e Padrões) · GoF (Padrões de Projeto) · Freeman & Freeman (Use a Cabeça! Padrões de Projetos) · Bloch (Java Efetivo)

★ = item marcado como importante nas anotações.

Material Auxiliar

Métodos Avançados de Programação

Como pensar sobre design de software?



1. Apresentação da disciplina e ideias iniciais

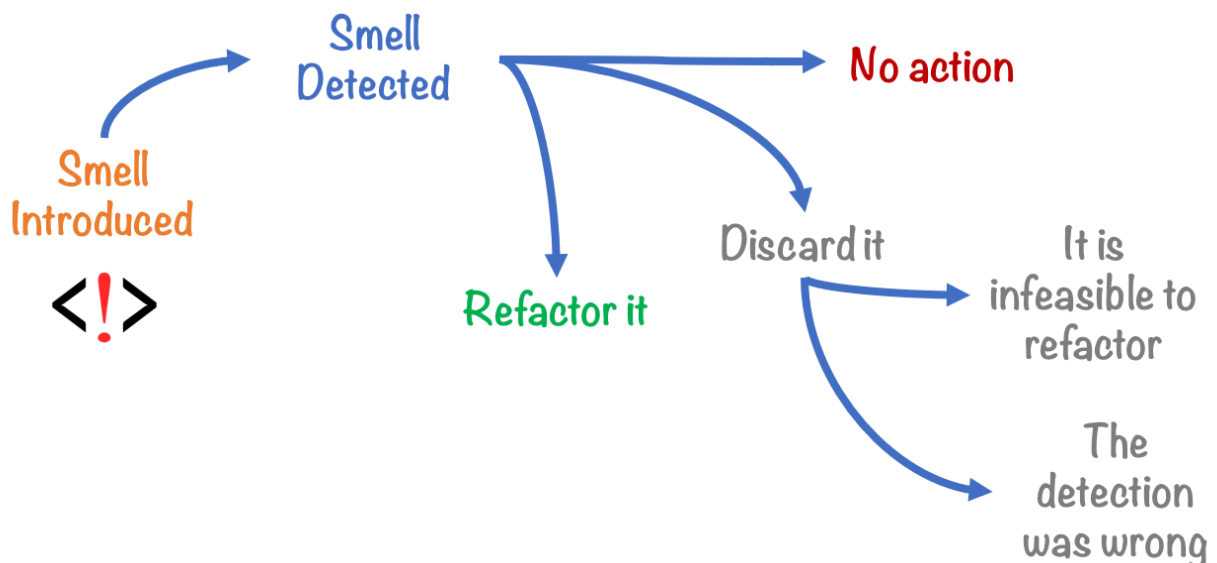
1.1 Sobre o que é a disciplina

- É a disciplina de **Métodos Avançados de Programação (orientada a objetos)**.
- **Objetivo:** estudar padrões de projeto para melhorar a forma de programar.
- O objetivo não é ensinar uma linguagem, e sim conceitos aplicáveis a qualquer linguagem orientada a objetos (os exemplos usam Java, mas a ideia vale para C#, Python, etc.).
- **Padrões são boas práticas, mas ninguém é obrigado a segui-los.** Eles ajudam quando há um problema real.
- APS e MAP estão dentro do campo mais geral da Engenharia de Software.

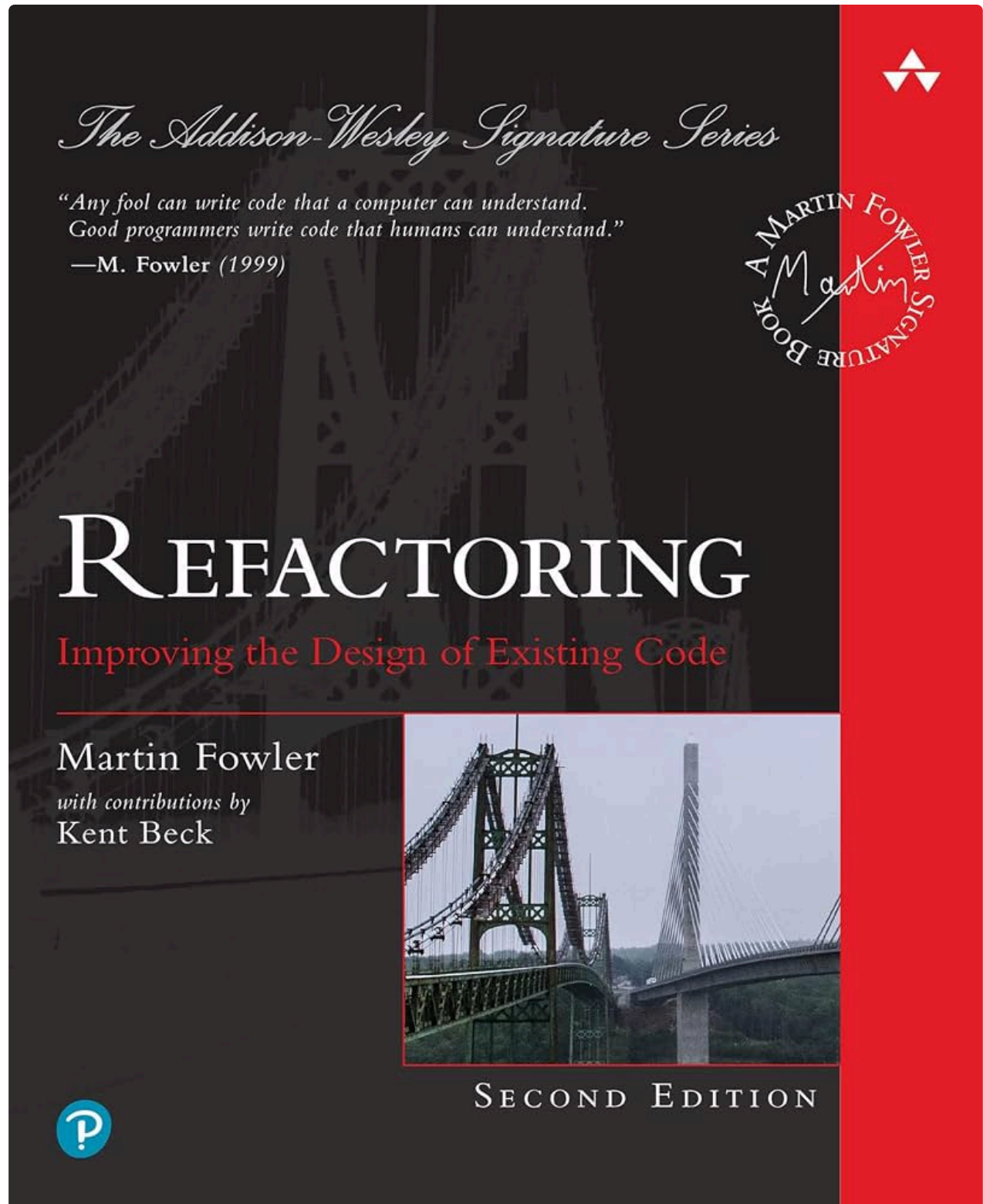
1.2 Mau cheiro no código (bad smell)



- **Mau cheiro (bad smell / code smell)** é um **sinal na estrutura do código** de que existe um problema de design. O código pode até **funcionar**, mas está **difícil de entender, mudar ou testar**.
- **"O que é um bom código?"**: um código com mau cheiro não é um bom código, ainda que passe nos testes.
- **Exemplos vistos:**
 - **Muitos if s** (quem tem muitos if s geralmente tem mau cheiro), como no `calcularTotal` da seção 2.
 - **Classe com várias responsabilidades sem relação** (baixa coesão), como a `Classe Utilitario`.
 - **Acoplamento alto**, em que uma mudança se espalha.
- **Refatorar é o "desodorante" que tira o bad smell**: refatorar é **reorganizar a estrutura interna do código sem alterar o que ele faz por fora**, eliminando o cheiro. Na disciplina, isso aparece na ementa (técnicas de refatoramento) e no ciclo real de desenvolvimento.



O termo *code smell* foi popularizado por Kent Beck e Martin Fowler (livro *Refactoring*)



2. Ponto de partida: um código que funciona, mas...

2.1 O exemplo da classe Pedido

Um código simples que calcula o total de um pedido de acordo com o tipo de pagamento:

```
class Pedido {  
    public double calcularTotal(String tipoPagamento) {
```

```

        if (tipoPagamento.equals("boleto")) {
            return 100 - 5;
        } else if (tipoPagamento.equals("cartao")) {
            return 100 - 2;
        }
        return 100;
    }
}

```

O método aplica um desconto de 5 no boleto e de 2 no cartão sobre uma base fixa de 100 (valores simplificados só para o exemplo). Qualquer outro tipo paga o valor cheio.

1. O que acontece se surgirem novos tipos de pagamento?
2. Onde isso vai quebrar?
3. Isso é fácil de manter?

2.2 Por que esse código é frágil

O código é **fácil de escrever, mas difícil de evoluir**. O raciocínio é o seguinte:

- Cada novo tipo de pagamento obriga a **editar o método existente**, acrescentando mais um `else if`.
- Mexer em código que já funciona traz o risco de **quebrar o que já estava correto** (por exemplo, alterar sem querer o cálculo do boleto ao incluir o Pix).
- Se a mesma verificação de tipo aparecer em outros pontos do sistema, todos precisariam ser alterados.
- O método cresce a cada variação e passa a concentrar regras que poderiam estar separadas.
- Um método cheio de `if s` é um **sinal típico de mau cheiro**

Observação técnica: lembrando em Java, comparar `String` com `==` compara referências, não conteúdo. Os trechos de código posteriores usam `tipo == "boleto"` apenas como pseudocódigo. Na prática, o certo é `.equals`.

3. O que é um "bom código"?

3.1 Os critérios possíveis

- Funciona?
- É rápido?
- É bonito?
- É fácil de mudar? ★

"O que importa mais?". IMPORTA QUE SEJA FÁCIL DE MUDAR

3.2 Bom código é código fácil de entender e de mudar

O motivo é que **software inevitavelmente muda**:

- **Requisitos mudam** (metodologia ágil: o que o cliente quer hoje não é o que vai querer amanhã);
- **Sistemas crescem** (novas funcionalidades se somam às antigas);
- **Erros são descobertos** e precisam ser corrigidos.

Por isso, **fácil de mudar/manter** (flexibilidade) é o critério que sustenta os demais: um código que funciona hoje, mas não aceita mudança, deixa de funcionar quando o contexto muda.

3.3 Design é comunicação

O código é escrito hoje para:

- o **próprio autor**, daqui a 3 meses;
- **outra pessoa** que vai ler e alterar;
- **um sistema maior**, no qual esse código vai ser integrado.

Por isso, **design é comunicação**: as decisões precisam estar claras para quem vem depois.

4. Projetando melhores soluções

Antes de escrever ou alterar código, quatro perguntas orientam o design:

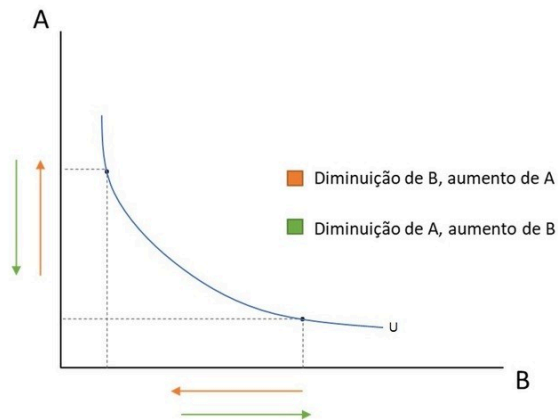
1. **Qual é o problema?** Sem entender o problema, qualquer solução é chute.
2. **O que pode mudar?** Identifica os pontos de variação (ex.: os tipos de pagamento).
3. **O que precisa ser flexível?** Nem tudo precisa; só o que tem chance real de variar.
4. **O que precisa ser simples?** O restante deve ficar direto, sem estrutura desnecessária.

4.1 Perguntas antes de mudar o código

Aplicadas ao exemplo do `if (tipo == "boleto") { ... } else if (tipo == "cartao") { ... }:`

- **Isso vai crescer?**
 - **Novos tipos vão aparecer?**
 - **Quem muda esse código?**
-

5. Design é sobre trade-offs



O projetista sempre fica **entre dois valores que puxam em direções opostas**:

Trade-off	Como entender
Simplicidade × flexibilidade	Quanto mais flexível, mais estrutura (interfaces, classes) e, em geral, menos simples.
Desempenho × abstração	Camadas de abstração podem ter um custo de desempenho.
Rapidez × qualidade	Entregar rápido tende a reduzir o cuidado com o design.

Não existe solução perfeita. Design é escolher o equilíbrio adequado ao contexto.

6. Pensamento de design

6.1 Código não é uma atividade isolada

Escrever código **não é uma atividade isolada**: é **organizar decisões**. Três perguntas guiam essa organização:

- Quem (calsse, objeto, método, etc.) sabe o quê?
- Quem faz o quê?
- Quem depende de quem?

6.2 Responsabilidades e colaboração ★

- **Responsabilidade**: o que um objeto se compromete a fazer ou a saber.
- **Colaboração**: quando o objeto **não tem** determinada responsabilidade, mas **sabe quem tem** e recorre a esse outro objeto.

7. Conceitos fundamentais: acoplamento, coesão e abstração

7.1 Acoplamento

Acoplamento mede o nível de dependência entre módulos/classes.

Quando o código é **fortemente acoplado**:

- uma mudança em uma classe **pode exigir mudanças em várias outras**;
- fica **mais difícil testar uma classe isoladamente**;
- os componentes ficam **mais difíceis de reutilizar** em outros contextos.

 **alto acoplamento dificulta mudanças e testes unitários.** Um teste unitário verifica uma classe isolada; se ela depende de várias outras, é preciso preparar (ou simular) todas elas para testá-la, e uma mudança em qualquer dependência pode quebrar o teste.

Alto × baixo acoplamento



- **Alto acoplamento:** UI, Pagamento, Usuário, Pedido, Estoque e Notificação se conectam **todos com todos**, por setas em vários sentidos. Qualquer alteração se espalha.
- **Baixo acoplamento:** cada componente se liga apenas ao **Pedido**, por meio de **interfaces** (os "encaixes" circulares da figura). As dependências ficam limitadas e explícitas.

7.2 Coesão

Coesão mede o quanto **as responsabilidades e os elementos de um módulo estão relacionados entre si**.

Quando o código tem **alta coesão**:

- cada classe tem **um propósito claro** ★;

- as responsabilidades são **fortemente relacionadas**;
- o código é **mais fácil de entender e manter**;
- as mudanças tendem a ficar **mais localizadas**.

✎ Um texto é coeso quando **não foge do seu tema/eixo discursivo**. Da mesma forma, uma classe é coesa quando **todas as suas responsabilidades giram em torno do mesmo assunto**. Exemplo: a classe `Pessoa` **não deve** incluir métodos de **gerar boleto**, pois isso é assunto de cobrança/pagamento, não de pessoa.

✎ **Alta coesão deixa o código mais simples**: cada classe fica pequena, com um propósito claro, o que facilita entender, testar e alterar.

Baixa × alta coesão



- **Baixa coesão:** Classe Utilitário com `salvarUsuario()`, `enviarEmail()`, `gerarRelatorio()`, `calcularDesconto()` e `conectarBanco()`. São muitas responsabilidades sem relação, o que torna a classe difícil de entender, manter e testar.
- **Alta coesão:** Classe RelatorioPedido com `gerarRelatorio()`, `filtrarPedidos()` e `imprimirRelatorio()`. Tudo gira em torno de uma única responsabilidade, então é fácil de entender, manter e reutilizar.

7.3 O papel das abstrações

As abstrações servem para:

- **esconder decisões** ★ (o "como" fica escondido atrás de um "o quê");
- **permitir mudança**;
- **não** para "deixar bonito".

Resultado: **flexibilidade e evolução sem quebrar tudo**.

Uma abstração permite depender **do que um objeto faz, e não de como ele faz**; e uma boa abstração tem **motivo claro para existir**, isolando uma **variação relevante**.

7.4 Como os três conceitos se relacionam

Juntos, favorecem objetos flexíveis e com responsabilidades bem definidas:

- **Baixo acoplamento:** reduz dependências entre objetos.
- **Alta coesão:** mantém responsabilidades relacionadas no mesmo objeto.
- **Abstração:** permite depender do que um objeto faz, não dos detalhes de como faz.
- **Objetivo:** limitar responsabilidades e reduzir dependências desnecessárias.

Mantra da POO: alta coesão e baixo acoplamento. Ao avaliar qualquer decisão de design, essas são as duas perguntas de referência.

Uma forma de reduzir acoplamento é fazer uma classe depender de **uma abstração, e não de uma implementação concreta:**

```
class PedidoService {  
    private Pagamento pagamento;  
  
    PedidoService(Pagamento pagamento) {  
        this.pagamento = pagamento;  
    }  
}
```

Aqui `Pagamento` pode ser uma **interface**. O ganho **não** está simplesmente em "usar uma abstração como parâmetro". Está em `PedidoService` **não precisar conhecer o tipo de pagamento** (`PagamentoPix`, `PagamentoCartao`, ...).

```
// a mesma classe atende tipos diferentes  
PedidoService a = new PedidoService(new PagamentoPix());  
PedidoService b = new PedidoService(new PagamentoCartao());
```

Encadeamento final:

- Bom código é fácil de mudar.
- Mudanças ficam difíceis com dependências excessivas.
- Baixo acoplamento limita essas dependências.
- Alta coesão mantém as responsabilidades focadas.
- Abstrações escondem detalhes e estabelecem **contratos estáveis** (interfaces).
- Objetos ficam **mais fáceis de substituir, testar e modificar**.

8. Quando abstrair e pensamento orientado à mudança

8.1 Quando abstrair?

Cada abstração deve ter **um motivo claro para existir**. Evitar abstrair:

- **Cedo demais:** cria-se uma estrutura que talvez nunca seja necessária.

- **Tarde demais:** o código já pode estar difícil de modificar.

Ideal: abstrair **quando se identifica uma variação relevante que precisa ser isolada.**

8.2 Pensamento orientado à mudança

Princípio: **separar o que varia daquilo que não varia.** O que provavelmente vai mudar deve ser **protegido com design.**

Exemplo voltando ao pagamento:

- **Hoje:** dois tipos de pagamento.
- **Amanhã:** dez tipos (boleto, cartão, pix, cash, carteira e combinações como boleto + cartão, boleto dividido, pix + cash, carteira + pix, ...).

Com `if/else`, cada combinação viraria mais um ramo. O design precisa **antecipar essa variação sem exagerar.**

Pensar **orientado à mudança** é **separar o que precisa mudar do que não precisa.**

8.3 Overengineering ★

Overengineering é **abstrair cedo demais**

- **Problema clássico:** abstrair cedo e usar padrão de projeto **sem necessidade.**
- **Resultado:** código complexo e difícil de entender.
- **Regra:** design bom **resolve um problema real.** Se não há problema, não há necessidade de padrão.
 - Padrões existem para resolver **problemas recorrentes.** Sem um problema claro (código difícil de mudar, muitos `if s`, acoplamento alto), aplicar um padrão só complica o código.

Exemplo com o Pedido

- Se o sistema tem **dois tipos de pagamento estáveis**, sem previsão de crescer, o `if/else` pode ser suficiente. Criar interface, várias estratégias, fábrica e registro seria **estrutura demais.**
- Se surgem **novos tipos e combinações**, o `if/else` passa a doer: cresce, quebra o que já funciona e concentra regras. Aí sim a abstração se justifica.
- Ou seja, o mesmo código pode ser adequado ou insuficiente dependendo do **contexto e da variação real.**

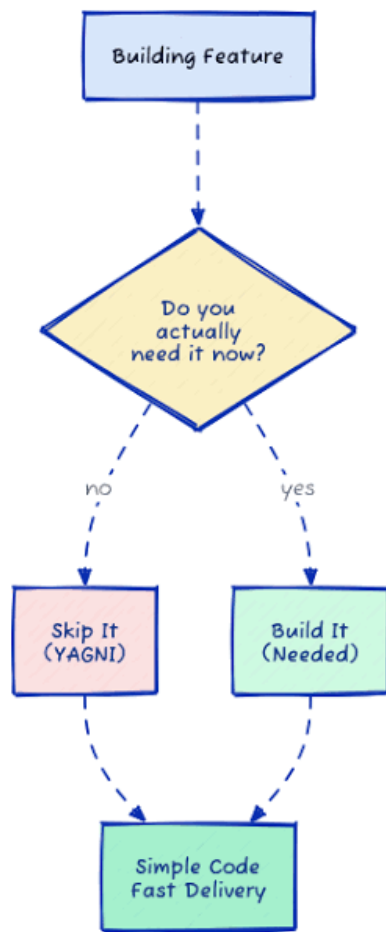
Os dois erros opostos

Erro	Consequência
Abstrair cedo demais (overengineering)	Complexidade sem benefício, código difícil de entender.
Abstrair tarde demais	Código já difícil de modificar, mudanças caras e arriscadas.

Como evitar

- Fazer as **perguntas da seção 8.4** (está acoplado demais? vai mudar? faz mais de uma coisa? está difícil de entender?).
- Abstrair **quando surge a variação relevante**, não "por precaução".
- Usar o **ciclo de refatoração**: entender o problema, refatorar e melhorar o design quando a dor aparece. **Parar é uma decisão consciente**: "bom o suficiente para agora".
- Lembrar que **se não há problema, não há necessidade de padrão**, e que **todo padrão traz trade-offs**.

Complemento: um princípio conhecido nessa linha é o **YAGNI** ("you aren't gonna need it": não implemente o que ainda não é necessário).



8.4 Como pensar ao codificar

Perguntas para fazer sempre:

- Isso está **acoplado demais**?
- Isso **vai mudar**?
- Isso está **fazendo mais de uma coisa**? (coesão)
- Isso está **difícil de entender**?

9. Onde estamos na disciplina

9.1 APS × MAP

Critério	Análise e Projeto de Sistemas (APS)	Métodos Avançados de Programação (MAP)
Nível de abstração	Alto	Baixo (design de código)
Foco	Modelagem do sistema, casos de uso, arquitetura geral, responsabilidades amplas	Organização de classes, relações entre objetos, decisões de implementação, evolução e manutenção
Pergunta que responde	Quais são os componentes do sistema e como eles interagem?	Como esse comportamento é implementado de forma flexível e sustentável?

9.2 GRASP × GoF (níveis de abstração)

Critério	GRASP	GoF (padrões de projeto)
Nível	Maior nível de abstração	Menor nível
Define	Quem deve fazer o quê	Como implementar
Foco	Responsabilidade e acoplamento	Estrutura e comportamento

- Um **não substitui** o outro: **se complementam**.
- **GRASP orienta decisões; GoF concretiza soluções**.
- **Ideia central da disciplina:** o sistema já se sabe projetar; agora se aprende a **projetar o código com qualidade**.

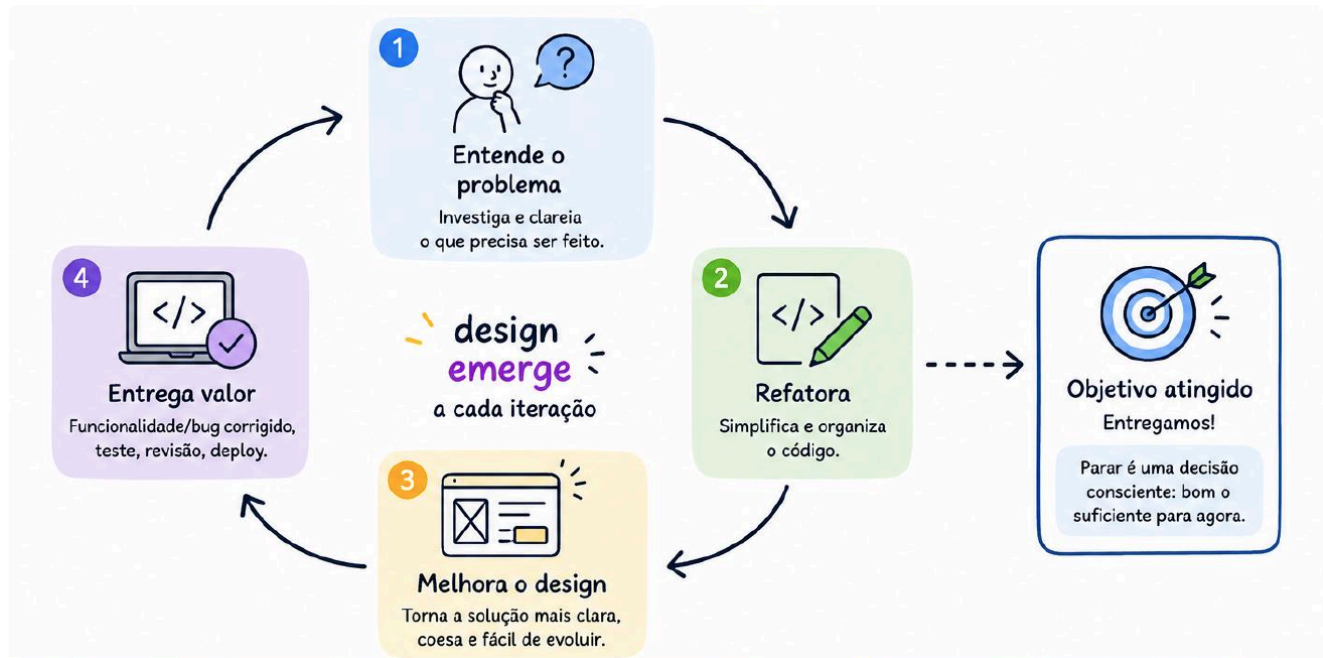
9.3 O que já se deve saber

- O que são **classes**?
 - O que são **objetos**?
 - O que são **propriedades**?
 - O que são **responsabilidades**?
 - O que são os **relacionamentos**?
 - **Como** os relacionamentos são definidos?
-

10. Ciclo real de desenvolvimento e mudança de mentalidade

10.1 Ciclo real de desenvolvimento

O ciclo é **iterativo**, **não linear**, e o **design emerge a cada iteração**. Quatro etapas:



Não é infinito

- O ciclo acontece várias vezes dentro de um objetivo (feature, bug, melhoria).
- Para-se quando o resultado atende ao contexto (**escopo, tempo, risco, valor**).
- Depois, um novo ciclo começa para o próximo objetivo.

Foco em entregar valor: itera-se para aprender, simplificar e evoluir. (Isso também protege contra o overengineering: parar quando é suficiente.)

10.2 Mudança de mentalidade 🧠

- **Antes:** "Qual é a arquitetura?" (ESW e APS)
- **Agora:** "Esse código vai continuar bom quando o sistema crescer?" (MAP)

11. Sobre padrões de projeto

- Cada padrão **descreve um problema que ocorre frequentemente** e então descreve o **cerne da solução**, de forma a poder reusá-la milhares de vezes em situações diferentes.
- Padrões são **soluções para problemas recorrentes, não receitas obrigatórias**.
- Funcionam como **vocabulário comum** para discutir soluções.

- Possuem **consequências**: perdas e ganhos associados à sua aplicação (por isso o cuidado com o overengineering).
 -  **Padrões são boas práticas, mas ninguém é obrigado a segui-los.**
 - **Na própria API do Java, vários padrões foram implementados.** Exemplos comuns:
 - Comparator passado a `List.sort(...)` ou `Collections.sort(...)`: **Strategy** (o algoritmo de comparação é trocável).
 - Iterator das coleções: **Iterator**.
 - `java.io` (por exemplo, `BufferedReader` envolvendo outro `Reader`): **Decorator**.
 - Componentes do Swing (um painel que contém outros componentes): **Composite**; e os *listeners* de eventos: **Observer**. A classe `java.util.Observable` é a versão legada e está depreciada desde o Java 9.
 - Os padrões descritos são **soluções gerais e reutilizáveis**, por isso aparecem em bibliotecas e frameworks que se usam no dia a dia, muitas vezes sem que se perceba.
-

12. Informações da disciplina

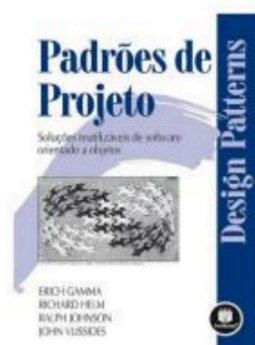
12.1 Ementa

- Técnicas avançadas de análise e projeto de software orientado a objetos empregados no desenvolvimento de software.
- Padrões de projeto.
- Técnicas de refatoramento de software.
- Conceitos sobre frameworks e desenvolvimento orientado a componentes.

12.2 Objetivos

- Apresentar o projeto orientado a objetos voltado à modelagem de soluções e sua implementação de software.
- Apresentar o conceito de padrões para o projeto e a implementação de soluções flexíveis.
- Possibilitar a construção de soluções de software usando padrões.

12.3 Referências



- Utilizando UML e Padrões, Craig Larman, 3ª ed., Bookman, 2017.
- Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos, Gamma, Helm, Johnson e Vlissides, Bookman, 2000.
 - **É o clássico inquestionável.** Tem uma leitura mais complexa e densa, mas foi justamente pensado para servir como um catálogo de consulta ao invés de um livro para ler de capa a capa
- Use a Cabeça! Padrões de Projeto, Eric Freeman e Elisabeth Freeman, 2ª ed., Alta Books, 2009.
 - Linguagem e leitura bem mais dinâmicas, porém não abrange todos os padrões
- Java Efetivo, Joshua Bloch, 3ª ed., Alta Books, 2018.

12.4 Avaliações

1ª unidade temática

- **1ª avaliação:** 07/10/2026 (teórica)
- **2ª avaliação:** 09/11/2026 (prática)
- **3ª avaliação:** roteiros de prática durante as aulas e atividades de pesquisa e apresentação
- **Reposição:** 14/11/2026 (prática) e 16/11/2026 (teórica)

2ª unidade temática

- **1ª avaliação:** 14/12/2026 (teórica)
- **Reposição:** 16/12/2026
- **Projeto:** entrega em 12/02/2027; apresentação em 15 e 17/02/2027

Final

- **22/02/2027**

12.5 Regras importantes

- Projetos só são aceitos se desenvolvidos dentro dos **temas divulgados**.
- **Não há reposição de projeto.** É preciso se organizar dentro do prazo.
- Em caso de **cópia** de projetos, a nota dos grupos é **dividida por 2**.
- A **reposição** é direito de quem falta no dia da prova (usar com sabedoria).
- Direito a **25% de ausências** sem reprovação por falta.

Resumo

DESIGN DE SOFTWARE (Aula 01)

|

|— PONTO DE PARTIDA

| |— Pedido.calcularTotal com if/else: fácil de escrever, difícil de evoluir

|

|— BOM CÓDIGO

| |— Funciona? Rápido? Bonito? FÁCIL DE MUDAR ★

| |— Porque requisitos mudam, sistemas crescem, erros aparecem

| |— Design é comunicação (você em 3 meses, outra pessoa, sistema maior)

|

|— PROJETANDO MELHORES SOLUÇÕES

| |— Qual o problema? O que muda? O que é flexível? O que é simples?

|

|— TRADE-OFFS (não existe solução perfeita)

| |— Simplicidade × flexibilidade

| |— Desempenho × abstração

| |— Rapidez × qualidade

|

|— PENSAMENTO DE DESIGN

| |— Quem sabe o quê? Quem faz o quê? Quem depende de quem?

| |— Responsabilidade + colaboração ★

|

|— MAU CHEIRO (BAD SMELL)

| |— Sinal de problema de design: muitos ifs, baixa coesão, acoplamento alto

| |— Refatorar: corrigir a estrutura sem mudar o comportamento

|

|— CONCEITOS

| |— Acoplamento: dependência entre módulos (quanto menor, melhor)

| |— Coesão: relação entre as responsabilidades (quanto maior, melhor)

| |— Abstração: esconder decisões, depender do "o quê" e não do "como"

|

|— ABSTRAIR NA HORA CERTA

| |— Nem cedo demais (overengineering) nem tarde demais

| |— Separar o que varia do que não varia

|

|— ONDE ESTAMOS

| |— APS: alto nível (componentes e interações)

| |— MAP: baixo nível (design de código)

| |— GRASP (quem faz o quê) + GoF (como implementar)

|

|— MANTRA DA POO: alta coesão e baixo acoplamento

|

|— PADRÕES: boas práticas (não obrigatórias); presentes na API do Java

|
└ MENTALIDADE: "esse código continua bom quando o sistema crescer?"